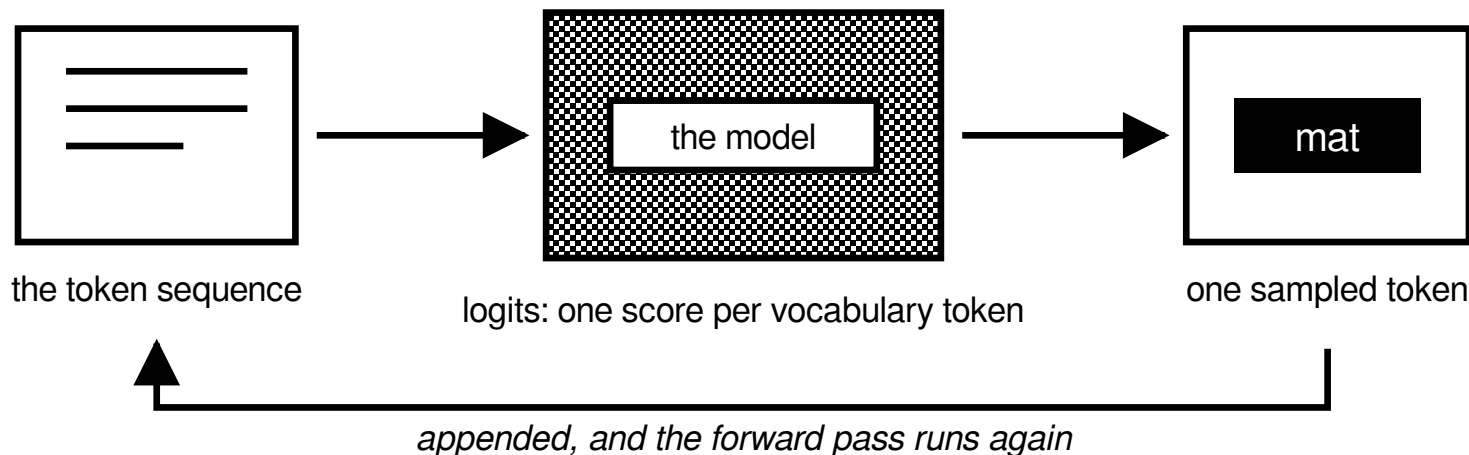


Language Models, Explained

What follows is a working picture of how a language model turns text into more text. It is coarse in places and nowhere false, and it is enough to reason about the thing rather than guess at it.

Each card cites where to read further. The last one asks a live model, if you have configured one. Press Next.



Jurafsky & Martin, Speech and Language Processing, 3rd ed. draft · web.stanford.edu/~jurafsky/slp3

Back

Next

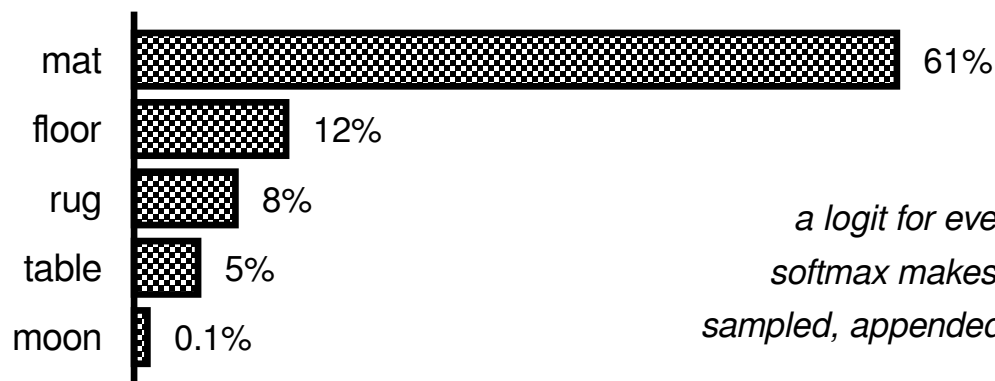
Start Over

It predicts the next token

The model reads a sequence of tokens and scores every token in its vocabulary — tens of thousands of numbers, one per candidate. A softmax turns those scores into a probability distribution, one token is drawn from it and appended, and the whole sequence goes through again.

That loop is the entire mechanic. Everything after this card follows from it.

The cat sat on the



*a logit for every token in the vocabulary;
softmax makes them a distribution, one is
sampled, appended, and the pass runs again.*

Vaswani et al., Attention Is All You Need, NeurIPS 2017 · [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)

Alammar, The Illustrated Transformer · jalammar.github.io/illustrated-transformer

Back

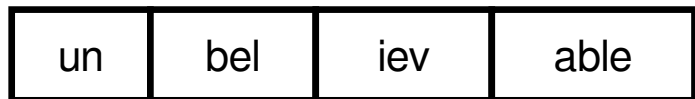
Next

Start Over

A token is not a word

A tokenizer splits text into subword units learned by byte-pair encoding, which merges the commonest adjacent pairs over and over. Frequent words survive whole while rare ones shatter, and a leading space usually belongs to the token after it.

The model never sees letters, so counting the r's in a word asks it about something it was never given.



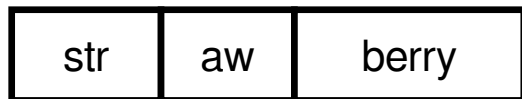
301

6789

12144

1122

four token ids, one word



three tokens, none of them the letter r

Byte-pair encoding merges the commonest pairs, so frequent words survive whole.

Asking how many r's are in "strawberry" asks about letters it was never shown.

Sennrich, Haddow & Birch, Neural Machine Translation of Rare Words with Subword Units, ACL 2016 · arXiv:1508.07909

Watch a tokenizer work: tiktokenizer.vercel.app

Back

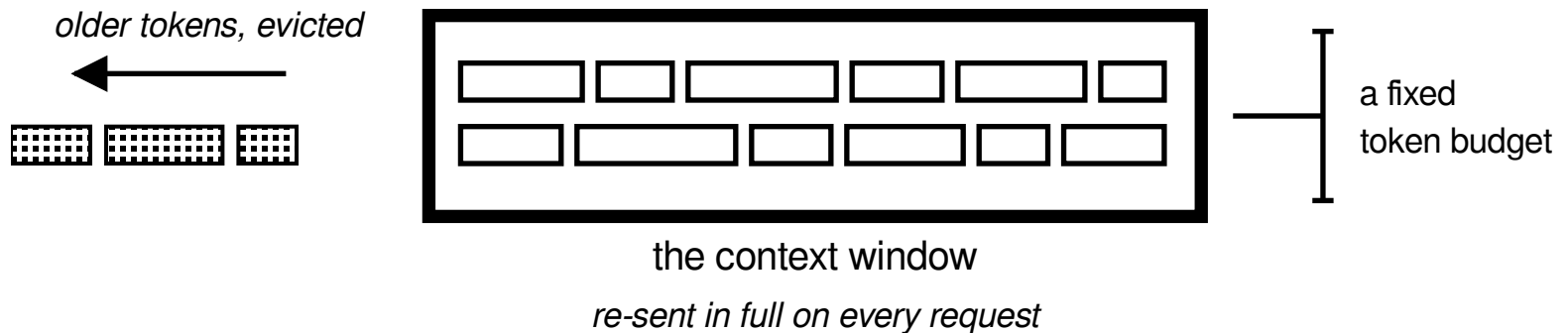
Next

Start Over

The context window is all it knows

Nothing persists between calls. The weights are frozen, so whatever a chat appears to remember about you was re-sent as text in that same request, and the window holding it has a hard limit in tokens.

Position inside that window matters: models use what sits at the start and the end of a long context far more reliably than what is buried in the middle.



Liu et al., Lost in the Middle: How Language Models Use Long Contexts, TACL 2024 ·
arXiv:2307.03172

Back

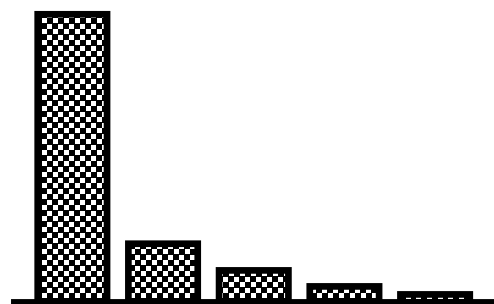
Next

Start Over

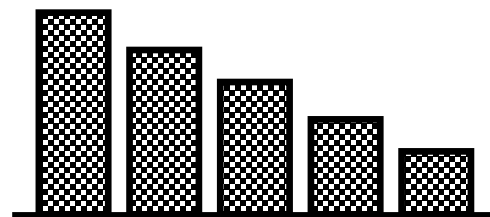
Temperature reshapes the distribution

Sampling is a stage after the model, and the scores it is handed do not change. Temperature divides them before the softmax: under one sharpens the distribution toward the top candidates, over one flattens it. Top-p keeps the smallest set of tokens whose probability sums to p .

Turning it down buys consistency. Truthfulness is not on this dial.



temperature 0.2
sharpened toward the mode



temperature 1.2
flattened; the tail is reachable

Holtzman et al., The Curious Case of Neural Text Degeneration, ICLR 2020 · arXiv:1904.09751

Back

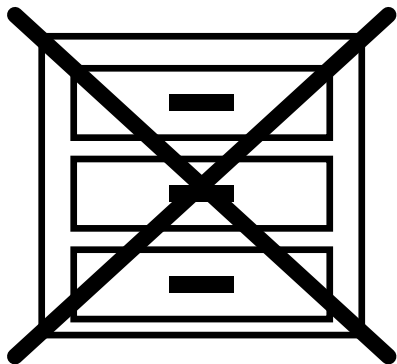
Next

Start Over

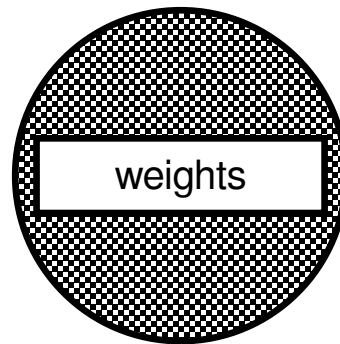
It is not looking anything up

There is no index and no retrieval step. An answer is computed in one forward pass from billions of weights that encode statistical regularities of the training data, so facts seen often come back well and facts seen rarely are reconstructed — with the same fluent cadence either way.

If it has to be right, put the source in the context and say so.



no index, no retrieval



one forward pass computes it

Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020 · arXiv:2005.11401

Back

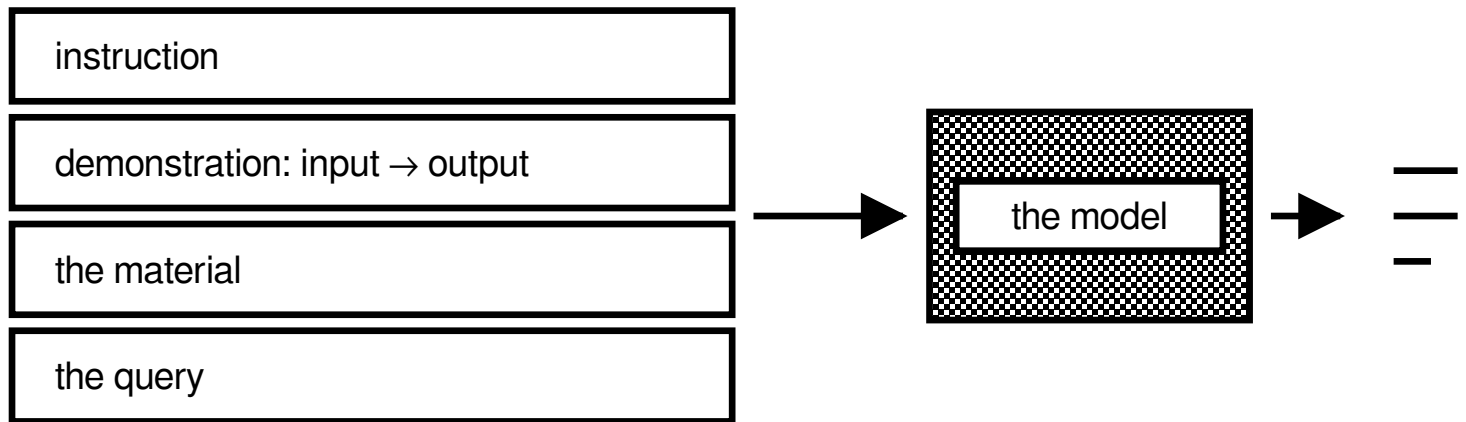
Next

Start Over

A prompt is just more context

Nothing in a prompt is a command. Your text conditions the same next-token distribution everything else conditions, which is why an incantation like “be accurate” moves almost nothing while one worked example moves a great deal: the behaviour was already there, and the example locates it.

Show the shape of the answer you want, then hand over the material.



one sequence; the model sees no seam between them

Brown et al., Language Models are Few-Shot Learners, NeurIPS 2020 · arXiv:2005.14165

Back

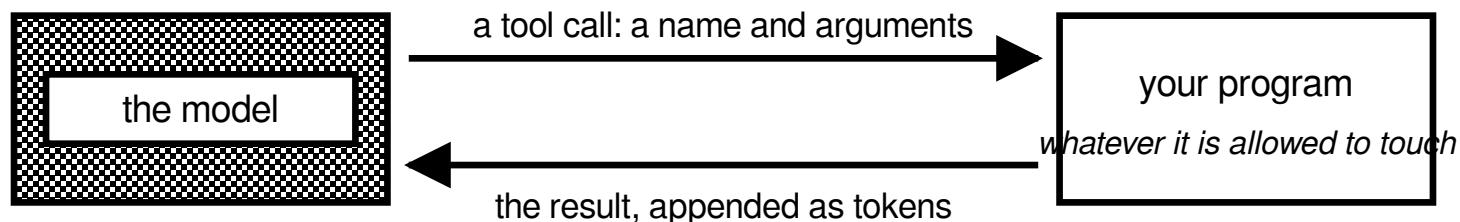
Next

Start Over

Tools: it asks, something else acts

A model emits tokens and nothing else. Given a schema of the calls available to it, it can emit one — a name and arguments — and stop. Your program parses that, does the work, and feeds the result back as more tokens.

HyperLab's assistant works this way, and each action it takes is a Command, which is why you can undo it.



the loop repeats until the model stops asking for tools, which is what an agent is

Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023 · arXiv:2210.03629

Back

Next

Start Over

Now ask a real one

Everything so far was written in advance. What comes back below will not be. Configure a provider under AI › Show Assistant › Settings, then press Ask.

With none configured it says so and the stack keeps working, which is a rule the runtime enforces.

Explain a context window to someone who has never heard of one.

Ask

Ouyang et al., Training language models to follow instructions with human feedback, NeurIPS 2022 · arXiv:2203.02155

Back

Next

Start Over

Everything cited on the way here, and a few places to go further than these cards can.

Jurafsky & Martin, Speech and Language Processing, 3rd ed. draft · web.stanford.edu/~jurafsky/slp3

Alammar, The Illustrated Transformer · jalammar.github.io/illustrated-transformer

Karpathy, nanoGPT — a GPT in about 300 lines · github.com/karpathy/nanoGPT

Elhage et al., A Mathematical Framework for Transformer Circuits, 2021 · transformer-circuits.pub

Bender, Gebru, McMillan-Major & Shmitchell, On the Dangers of Stochastic Parrots, FAccT 2021

Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, NeurIPS 2022

· [arXiv:2201.11903](https://arxiv.org/abs/2201.11903)

Ji et al., Survey of Hallucination in Natural Language Generation, ACM Computing Surveys ·

[arXiv:2202.03629](https://arxiv.org/abs/2202.03629)

Schick et al., Toolformer: Language Models Can Teach Themselves to Use Tools, 2023 ·

[arXiv:2302.04761](https://arxiv.org/abs/2302.04761)

Back

Next

Start Over